PROJECT PHASE I REPORT

ON

BROWSEBLOCK

Submitted by

Deon George (SJC23CS086)

Amal Kurian Royce (SJC23CS037)

Maria Royce (SJC23CS159)

Melbin K Thomas (SJC23CS164)

to

the APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the degree

of

Bachelor of Technology

in

Computer Science and Engineering



Department of Computer Science and Engineering

St. Joseph's College of Engineering and Technology, Palai

(Autonomous)

October : 2026

Declaration

We undersigned hereby declare that the project phase 1 report on **"BrowseBlock: AI Browser Extension for Real-Time Browser-Based Attack Detection and Prevention"**, submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala, is a bonafide work done by us under the supervision of **Prof. Divya Sunny**. This submission represents our ideas in our own words and where ideas or words of others have been included. We have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Name and Signature of Student

Deon George (SJC23CS086)
Amal Kurian Royce (SJC23CS037)
Maria Royce (SJC23CS159)
Melbin K Thomas (SJC23CS164)

Place: Choondacherry
Date: 17-10-2026

ST. JOSEPH'S COLLEGE OF ENGINEERING AND TECHNOLOGY, PALAI

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the report entitled **"BrowseBlock: AI Browser Extension for Real-Time Browser-Based Attack Detection and Prevention"** submitted by **Deon George (SJC23CS086), Amal Kurian Royce (SJC23CS037), Maria Royce (SJC23CS159), and Melbin K Thomas (SJC23CS164)** to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record of the project work carried out by them under my guidance and supervision.

Project Guide
Prof. Divya Sunny
Assistant Professor
Department of CSE

Project Coordinator
Prof. Smitha Jacob
Assistant Professor
Department of CSE

Head of Department
Dr. Joby P P
Professor & Head
Department of CSE

Place : Choondacherry
Date : 17-10-2026

Acknowledgement

The success of this Phase I project required the guidance and assistance of many people, and we are privileged to have received their support throughout its completion. We sincerely thank everyone whose supervision, encouragement, and technical assistance made this work possible.

We would like to express our respect and gratitude to the management of St. Joseph's College of Engineering and Technology for providing us with the opportunity and platform to work on this project.

A special word of thanks goes to our beloved Principal, **Dr. V. P. Devassia**, for providing invaluable support and necessary facilities to carry out this project.

We are extremely indebted to **Dr. Joby P. P**, Professor & Head of the Department of Computer Science and Engineering, for his valuable suggestions and encouragement throughout the course of this project work.

We would also like to express our gratitude to our project coordinator, **Prof. Smitha Jacob**, Assistant Professor in the Department of Computer Science and Engineering, for her valuable suggestions and guidelines during the entire duration of this project. We are truly appreciate her contributions and technical support in preparing this report.

Our heartfelt thanks go to our project guide, **Prof. Divya Sunny**, Assistant Professor in the Department of Computer Science and Engineering, for her keen interest, guidance, and technical support throughout the project.

We are thankful and fortunate enough to have received constant encouragement, support, and guidance from all the staff members of the Department of Computer Science and Engineering. Their assistance played a crucial role in the successful completion of our Project Phase 1 work.

Abstract

BrowseBlock is an AI/ML-powered browser security platform designed to detect and help prevent browser-based attacks in real time. A Manifest V3 Chromium extension monitors supported navigation, webpage scripts, redirects, frames, notifications, downloads, email content, and observable extension activity. A React dashboard supports authenticated manual scans of URLs, QR codes, emails, and files, together with threat reports, explanations, and analytics. The FastAPI backend validates requests and coordinates XGBoost URL analysis, Random Forest email screening, static security rules, QR decoding, file inspection, and RAG/LLM-assisted email analysis. Firebase Authentication manages dashboard identity, while the backend remains the controlled boundary for protected operations; the finalized design does not require a separate application database. The proposed system covers URL and fake-QR phishing, cross-site scripting, credential harvesting, malicious redirects, clickjacking, cryptojacking, malicious downloads, email phishing, notification scams, and browser-extension abuse. Each result is classified as Safe, Suspicious, or High-risk and includes evidence, analysis completeness, and the actual protective action. The Phase I work establishes the requirements, architecture, methodology, literature foundation, and evaluation plan for a privacy-conscious security layer that complements existing browser and antivirus protections.

Contents

Declaration ii

Acknowledgement iv

Abstract v

List of Figures ix

1 Introduction 1

1.1 Background . . . 1

1.2 Problem Statement . . . 1

1.3 Objectives of the Project . . . 2

1.3.1 User Authentication and Access Control . . . 2

1.3.2 Event-Driven Browser Monitoring . . . 2

1.3.3 Threat Detection and Classification . . . 2

1.3.4 Evidence-Based Explanations . . . 3

1.3.5 Warnings and Protective Actions . . . 3

1.3.6 Dashboard Analysis and Reporting . . . 3

1.3.7 Privacy-Preserving Analysis . . . 3

1.4 Scope and Limitations . . . 3

1.4.1 Scope . . . 3

1.4.2 Limitations . . . 4

1.5 Significance of the Study . . . 4

1.6 Project Overview . . . 4

1.7 Expected Outcomes . . . 5

1.8 Sustainable Development Goals . . . 5

1.8.1 SDG 4: Quality Education . . . 5

1.8.2 SDG 8: Decent Work and Economic Growth . . . 6

1.8.3 SDG 9: Industry, Innovation and Infrastructure . . . 6

1.8.4 SDG 16: Peace, Justice and Strong Institutions 6
1.8.5 SDG 17: Partnerships for the Goals 7

List of Abbreviations **1**

2 Literature Review **8**

2.1 XGBoost-Based URL Phishing Detection Method With Cross-Dataset Validation, 2026 8
2.2 Classification of Phishing Email Using Random Forest Machine Learning Technique, 2014 9
2.3 Detecting DOM-Sourced Cross-Site Scripting in Browser Extensions, 2017 10
2.4 Hidden Path: Understanding the Intermediary in Malicious Redirections, 2022 11
2.5 Implementation of Extension for Browser to Detect Vulnerable Elements on Web Pages and Avoid Clickjacking, 2016 12
2.6 Analyzing In-Browser Cryptojacking, 2024 13
2.7 Hidost: A Static Machine-Learning-Based Detector of Malicious Files, 2016 14
2.8 When Push Comes to Ads: Measuring the Rise of (Malicious) Push Advertising, 2020 15
2.9 CPDetector: Automated Detection of Excessive Permissions in Chrome Extensions, Preprint 16
2.10 A Login Page Transparency and Visual Similarity-Based Zero-Day Phishing Defense Protocol, 2025 17

3 Requirement Analysis **18**

3.1 Software Requirements 18
3.2 User Classes and Characteristics 18
3.2.1 Functional Requirements 18
3.2.2 Non-Functional Requirements 20
3.3 Design and Implementation Constraints 21
3.4 External Interface Requirements 21
3.4.1 User Interfaces 21

3.4.2 Software Interfaces . . . 21
3.4.3 Operating Environment . . . 21
3.4.4 Communication and Data Interfaces . . . 22
3.5 System Features . . . 22
3.5.1 URL and Fake QR Phishing . . . 22
3.5.2 Cross-Site Scripting . . . 22
3.5.3 Credential Harvesting and Redirects . . . 23
3.5.4 Clickjacking and Cryptojacking . . . 23
3.5.5 Malicious File Downloads . . . 24
3.5.6 Email Phishing . . . 24
3.5.7 Browser Notification Scams and Extension Abuse . . . 24
3.6 Risk Classification . . . 25
3.7 Logging, Audit, and Reporting Requirements . . . 25
3.8 User Documentation and Future Requirements . . . 25
3.9 Proposed Solution . . . 26
3.10 Scope and Limitations . . . 26

4 System Architecture 27
4.1 Architecture Diagram . . . 27
4.2 Activity Diagram . . . 29
4.3 ER Diagram . . . 31
4.4 Use Case Diagram . . . 33

5 Conclusion 35

References 36

List of Figures

4.1 BrowseBlock architecture diagram 28
4.2 BrowseBlock activity diagram 30
4.3 BrowseBlock entity relationship diagram 32
4.4 BrowseBlock use-case diagram 34

Chapter 1

Introduction

1.1 Background

Modern web browsers are used for banking, shopping, communication, education, file exchange, and access to cloud applications. This convenience also makes the browser an important attack surface. A user may encounter a phishing URL in a message, a QR code that redirects to a counterfeit website, a malicious download, a deceptive login form, an injected script, or a notification that imitates a security warning. These attacks often depend on a sequence of browser events rather than on one isolated file or address.

Traditional protection is distributed across browser warnings, blocklists, antivirus software, email filters, and individual security extensions. These controls are valuable, but they may not cover all browser activity, may provide little explanation, and may not distinguish between a detected threat and an action that was actually blocked. A URL may be new and absent from a blocklist, while a page may appear normal until its scripts, frames, redirects, or forms are examined.

BrowseBlock proposes an additional, explainable protection layer. It combines event-driven browser monitoring with machine-learning classification, static rules, reputation evidence, webpage inspection, and authorized language-model-assisted analysis. The design is intentionally modular so that each detector can be evaluated independently and the user can see both the evidence and the limitations of a result.

1.2 Problem Statement

Users need timely protection against browser-based attacks without manually investigating every URL, message, webpage, download, or browser extension. Existing solutions are

commonly specialized: URL classifiers may not analyze page behavior, email filters may not follow extracted links, and heavyweight dynamic analysis is unsuitable for continuous execution inside a browser extension. A blocklist can also miss newly created or modified campaigns.

The problem addressed by BrowseBlock is the design of a unified security platform that observes supported browser events, selects the relevant detector, combines available evidence, explains the resulting risk, and recommends or applies a protective action where browser capabilities permit. The system must also represent incomplete analysis explicitly. A timeout, unsupported format, unavailable browser event, or failed parser must not be reported as proof that an item is safe.

1.3 Objectives of the Project

The objectives are organized around the complete protection workflow.

1.3.1 User Authentication and Access Control

Provide sign-in and sign-out through Firebase Authentication, verify Firebase ID tokens at the FastAPI backend, and restrict reports, analytics, configuration, and other protected operations to authorized users and roles.

1.3.2 Event-Driven Browser Monitoring

Monitor supported navigation, webpage content, scripts, frames, redirects, notifications, downloads, email content, and observable extension activity without running every detector on every page. The triggering event should determine the smallest relevant analysis workflow.

1.3.3 Threat Detection and Classification

Detect ten threat categories: URL and fake-QR phishing, XSS, credential harvesting, HTML/JavaScript redirects, clickjacking, cryptojacking, malicious downloads, email

phishing, browser notification scams, and browser-extension abuse. Each result shall be classified as Safe, Suspicious, or High-risk with supporting indicators.

1.3.4 Evidence-Based Explanations

Show the user why an item was flagged, including suspicious URL features, form destinations, redirect evidence, script indicators, file characteristics, email signals, notification content, or permission mismatches. The explanation shall identify uncertainty and distinguish evidence from a guaranteed conclusion.

1.3.5 Warnings and Protective Actions

Warn the user and, where supported by browser APIs, restrict navigation, cancel a download, or provide another configured response. The system shall record whether the requested action was completed, unavailable, cancelled, or not applicable.

1.3.6 Dashboard Analysis and Reporting

Provide a React dashboard for manual URL, QR-code, email, and supported-file scans. The dashboard shall display results, evidence, threat details, analytics, and authorized history or reports stored through the backend.

1.3.7 Privacy-Preserving Analysis

Collect only the content required for the selected detector, never request real passwords, avoid unnecessary retention of raw content, and keep model credentials, database connections, and authentication secrets away from client code.

1.4 Scope and Limitations

1.4.1 Scope

The scope includes a Manifest V3 Chromium extension, a React and Tailwind CSS dashboard, a Python FastAPI backend, Firebase Authentication, MongoDB application data,

detection services, and optional AI, RAG, LLM, and reputation integrations. The system covers lightweight real-time checks as well as deeper user-authorized dashboard analysis. It includes a common result format, risk explanation, logging, reporting, and evaluation procedures.

1.4.2 Limitations

Protection is bounded by browser permissions, Chromium API visibility, network availability, model quality, detector coverage, external-service availability, and the evidence exposed by a page or file. A warning after navigation is not equivalent to prevention before page execution. Static analysis cannot establish every runtime behavior, and a low-risk result cannot guarantee that an item is harmless. The prototype will therefore report its analysis boundary and will complement rather than replace antivirus, browser security controls, and enterprise monitoring.

1.5 Significance of the Study

BrowseBlock connects detection, explanation, and user action in one workflow. General users receive understandable warnings, while security analysts and organizations can inspect evidence and reports. The project is significant because it treats privacy, uncertainty, and enforcement status as first-class security requirements rather than presenting a model score as an unconditional verdict.

1.6 Project Overview

The extension observes supported browser events and performs lightweight checks or forwards validated input to FastAPI. The backend authenticates protected requests, dispatches detector modules, combines model and rule evidence, stores authorized scan records in MongoDB, and returns a consistent result. The dashboard uses the same API to submit deliberate scans and review detailed reports.

Browser event $\rightarrow$ *Extension* $\rightarrow$ *FastAPI* $\rightarrow$ *Detection service* $\rightarrow$ *Risk result* $\rightarrow$ *Warning, report, or supported protective action*

1.7 Expected Outcomes

The Phase I outcome is a detailed requirements and design foundation for a modular BrowseBlock prototype. It defines threat modules, interfaces, data entities, authentication boundaries, privacy constraints, and evaluation metrics. Phase II will implement selected modules, train and validate models on representative data, test browser compatibility, measure precision, recall, false positives, latency, resource usage, and verify whether advertised warning, blocking, cancellation, quarantine, and logging actions were actually completed.

1.8 Sustainable Development Goals

BrowseBlock supports the Sustainable Development Goals (SDGs) by applying responsible computing to a practical cybersecurity problem. The project is primarily connected with SDG 4, SDG 8, SDG 9, SDG 16, and SDG 17. These connections are not claims that the software independently achieves an SDG; rather, they describe how the project contributes to the knowledge, safety, innovation, and responsible digital practices promoted by the goals.

1.8.1 SDG 4: Quality Education

BrowseBlock supports quality education by helping students and other learners use online learning platforms, digital libraries, email, and educational resources more safely. The extension provides understandable warnings and evidence instead of presenting only a technical model score. This can improve security awareness and help users recognize phishing, malicious redirects, unsafe downloads, deceptive notifications, and suspicious browser extensions. The project also provides an educational example of how machine learning, web technologies, privacy requirements, and secure software design can be combined in a real-world application.

1.8.2 SDG 8: Decent Work and Economic Growth

Unsafe links, credential theft, malicious files, and browser-based scams can cause financial loss and interrupt the work of individuals and small organizations. By identifying suspicious activity early and recommending appropriate protective actions, BrowseBlock can reduce avoidable exposure during online work. Its lightweight extension-based workflow is intended to support secure browsing without requiring expensive enterprise security infrastructure, making the approach relevant to users and organizations with limited resources.

1.8.3 SDG 9: Industry, Innovation and Infrastructure

The project contributes to SDG 9 through an innovative and modular security architecture. A Manifest V3 extension, FastAPI services, machine-learning detectors, reputation checks, a React dashboard, Firebase Authentication, and MongoDB are connected through defined interfaces. This architecture supports incremental improvement: new detectors, evidence sources, or evaluation methods can be added without redesigning the complete platform. The project also encourages reliable digital infrastructure by treating availability, latency, resource limits, and incomplete analysis as explicit design concerns.

1.8.4 SDG 16: Peace, Justice and Strong Institutions

Digital safety and trust are important components of inclusive and accountable institutions. BrowseBlock supports this goal by protecting users from common online deception and by recording evidence, risk classifications, explanations, and protective-action status in an auditable result model. Authentication, authorization, privacy controls, and clear reporting boundaries help prevent unauthorized access to scan history and reports. The system is designed to communicate uncertainty honestly, so that a failed or incomplete analysis is not incorrectly represented as proof that content is safe.

1.8.5 SDG 17: Partnerships for the Goals

Effective cybersecurity depends on cooperation among developers, researchers, educators, users, institutions, and providers of threat intelligence or reputation services. BrowseBlock is designed with integration boundaries for external data sources, authorized AI-assisted analysis, model evaluation, and future collaboration. The project documentation, common result format, and explicit detector limitations can support responsible sharing of findings and future development while protecting credentials and unnecessary user content.

Chapter 2

Literature Review

2.1 XGBoost-Based URL Phishing Detection Method With Cross-Dataset Validation, 2026

The paper by Milosz Misiek and Tomasz Hyla presents a machine-learning approach for detecting phishing URLs using XGBoost. The study addresses the difficulty of identifying malicious URLs that resemble legitimate web addresses and the challenge of maintaining detection accuracy when the model is evaluated on datasets different from those used during training. The authors emphasize that URL characteristics vary across datasets, which can affect the generalization ability of phishing-detection models.

The proposed method uses Python-based feature extraction to derive URL characteristics, including lexical features, DNS and network information, security-related attributes, domain reputation, and redirect behavior. The study extracts more than 90 candidate features and applies feature selection to identify characteristics that remain useful across datasets. An XGBoost classifier is then evaluated using three datasets and compared with neural-network approaches. The paper reports 90.7% accuracy on its custom dataset and 77.8% average accuracy across three independent test datasets. These results highlight the importance of feature selection and cross-dataset evaluation.

BrowseBlock adopts XGBoost for URL phishing classification and feature-based URL analysis. The extracted URL characteristics are supplied to the trained classifier to determine whether a destination appears legitimate or suspicious. The study's emphasis on cross-dataset validation is relevant when evaluating BrowseBlock against previously unseen URLs. For QR-code phishing, BrowseBlock adds a decoding stage that extracts the embedded URL and sends it through the same URL-analysis pipeline; QR decoding itself is not the principal method proposed in this paper.

2.2 Classification of Phishing Email Using Random Forest Machine Learning Technique, 2014

The paper by Andronicus A. Akinyelu and Aderemi O. Adewumi investigates the use of machine learning to classify phishing emails. It addresses limitations of traditional email-filtering methods, which may fail to recognize newly developed phishing patterns because attackers frequently change the wording and structure of their messages. The research aims to identify useful email characteristics that distinguish phishing messages from legitimate messages while reducing the number of features required for classification.

The proposed approach extracts phishing-related features from a dataset of 2,000 emails and converts them into feature vectors for machine-learning classification. The authors use Random Forest and evaluate different feature combinations, including a reduced set of eight prominent features. The paper reports 99.7% classification accuracy for its evaluated model. This result reflects the dataset and evaluation procedure used in the study and should not be assumed to represent the accuracy of every phishing-detection system.

BrowseBlock adopts Random Forest as the initial email-phishing screening technique. The classifier evaluates available email features and identifies messages that require further analysis. BrowseBlock extends this approach by examining URLs extracted from email content, incorporating SPF, DKIM, and DMARC results when available, and using RAG with an LLM for deeper contextual analysis. These additional stages are part of BrowseBlock's proposed hybrid pipeline rather than technologies introduced by the Random Forest paper.

2.3 Detecting DOM-Sourced Cross-Site Scripting in Browser Extensions, 2017

The paper by Jinkun Pan and Xiaoguang Mao investigates DOM-sourced Cross-Site Scripting vulnerabilities in browser extensions. Extension scripts can interact with webpage content and may possess privileges beyond those available to ordinary webpage scripts. When attacker-controlled webpage data is processed unsafely by an extension, it may cause malicious script execution or unauthorized access to protected resources. The research focuses on identifying these vulnerabilities, particularly in user scripts associated with the Greasemonkey browser extension.

The authors propose a hybrid analysis framework with two phases. The first phase performs lightweight static analysis using text filtering and an Abstract Syntax Tree parser to identify potentially vulnerable code. The second phase applies dynamic symbolic execution together with a shadow-DOM component and constraint-solving techniques to investigate candidate vulnerabilities. The paper reports 58 previously unknown DOM-sourced XSS vulnerabilities in Greasemonkey user scripts. The results demonstrate the usefulness of combining lightweight source analysis with deeper investigation of potentially unsafe execution paths.

BrowseBlock adopts the concept of examining suspicious data flows between untrusted webpage inputs and potentially dangerous JavaScript operations. Its XSS module can inspect accessible scripts and identify indicators involving sources such as URL parameters and sinks such as unsafe DOM-manipulation functions. The paper provides a reference for more advanced static and dynamic analysis techniques; BrowseBlock should claim AST analysis, symbolic execution, or shadow-DOM analysis only if those techniques are actually implemented.

2.4 Hidden Path: Understanding the Intermediary in Malicious Redirections, 2022

The paper by Yuwei Zeng and co-authors examines how attackers use intermediary websites and redirect chains to conceal malicious destinations. Malicious redirects can make a user's navigation path difficult to understand by routing the browser through several domains before reaching the final page. Attackers may use delays, fabricated user interactions, or visitor characteristics to avoid detection by conventional security checks. The study investigates mechanisms used in these campaigns and the role of intermediate domains.

The researchers develop an iterative framework that uses a bipartite graph-based method to discover domains potentially associated with malicious redirections from real-world DNS traffic. They then use a Chrome-based crawler to visit suspicious domains and reconstruct redirect chains from browser performance logs. The analysis examines HTTP-, HTML-, and JavaScript-based navigation and identifies redirection fluxing, in which randomness is introduced into the redirect process to make malicious destinations harder to track.

BrowseBlock adopts the concepts of monitoring navigation changes, examining suspicious redirect chains, and assessing the final destination using URL analysis. The redirect module can evaluate the requested URL and subsequent destinations where the required navigation information is available, and submit suspicious destinations to the XGBoost URL detector. The paper's graph-based DNS mining and large-scale crawling framework are reference techniques; they must not be described as implemented BrowseBlock components unless separately developed.

2.5 Implementation of Extension for Browser to Detect Vulnerable Elements on Web Pages and Avoid Clickjacking, 2016

The paper by Dipti Pawade, Abhilasha Lahigude, Divya Reja, and Era Johri presents ClickProtect, a browser extension designed to identify webpage elements vulnerable to clickjacking. Clickjacking tricks a user into clicking an element different from the one intended. The paper examines invisible iframe overlays and fake cursors that conceal the actual target of a user interaction, with the objective of detecting suspicious elements and warning the user before an unsafe interaction.

The proposed system consists of a revealing unit and an extenuation unit. The revealing unit inspects webpage source code and presentation characteristics to identify potentially hidden or misleading elements. The extenuation unit makes suspicious elements visible or otherwise helps prevent the user from interacting with a concealed target. This separation is relevant to BrowseBlock because detection and user protection are separate responsibilities even when they are presented in one browser workflow.

BrowseBlock adopts the concepts of inspecting accessible webpage elements for suspicious visibility, overlay, and interaction indicators and warning the user when findings are concerning. These checks contribute to its clickjacking detection module, which can present supporting evidence through the extension popup and dashboard. Since legitimate websites also use iframes and overlays, the indicators should be combined with contextual checks rather than treating every iframe or invisible element as proof of an attack.

2.6 Analyzing In-Browser Cryptojacking, 2024

The paper by Muhammad Saad and David Mohaisen investigates in-browser cryptojacking, in which a webpage executes JavaScript that uses a visitor's computing resources to mine cryptocurrency without appropriate consent. Unlike conventional malware requiring a separate executable, in-browser cryptojacking operates through scripts loaded by a webpage. It can consume CPU resources, drain batteries, and affect normal system performance.

The researchers combine static and dynamic analysis. Static analysis extracts code-related features, while dynamic analysis investigates mining effects on CPU usage and battery consumption. The paper evaluates Logistic Regression, Linear Discriminant Analysis, k-Nearest Neighbours, Support Vector Machines, and Random Forest to distinguish cryptojacking scripts from benign and other malicious JavaScript samples. It also examines browser communication and the economic feasibility of cryptojacking as an alternative to online advertising.

BrowseBlock adopts static JavaScript inspection, suspicious-script feature extraction, and machine-learning-based classification for cryptojacking detection. Resource-usage indicators may also help when the required information is available, but CPU consumption alone cannot prove cryptojacking. BrowseBlock's claims must therefore reflect the script indicators and runtime information actually accessible to its browser extension.

2.7 Hidost: A Static Machine-Learning-Based Detector of Malicious Files, 2016

The paper by Nedim Šrndic and Pavel Laskov introduces Hidost, a static machine-learning system for detecting malicious files. It addresses malware hidden inside document formats such as PDF and SWF. These files are commonly legitimate, but their complex internal structures can be exploited through obfuscation, encoding, or manipulation of document elements. The research aims to improve detection without execution-based analysis.

Hidost combines information about a file's logical structure with its content to generate features for machine-learning classification. It performs static analysis without executing the file and was evaluated on datasets containing approximately 440,000 PDF files and 40,000 SWF files. Structural information characterizes files and improves detection of malicious documents. The reported experiments showed strong detection performance, particularly for malicious PDF files, compared with the evaluated antivirus engines.

BrowseBlock adopts static file analysis, file-structure and content-feature extraction, and machine-learning-based malicious-file classification. These methods can assess supported downloads without executing them. The proposed BrowseBlock file pipeline can combine static features with SHA-256 reputation checks and a trained classifier, if those components are implemented. SHA-256 matching is an additional reputation mechanism, not the central technique introduced by Hidost.

2.8 When Push Comes to Ads: Measuring the Rise of (Malicious) Push Advertising, 2020

The paper by Karthika Subramani and co-authors examines web push notifications used to deliver advertisements, including malicious campaigns. Web push notifications can reach users even when the original webpage is no longer in the foreground. Although legitimate applications use notifications, attackers can distribute deceptive advertisements and direct users to scam pages. The study investigates the prevalence of malicious push advertising and the limitations of traditional ad-blocking and URL-filtering techniques.

The authors develop PushAdMiner, which collects and analyzes web push notifications. An instrumented Chromium browser inspects service-worker behavior, while a crawler visits websites, registers for notifications, and collects messages. The system groups related notifications into advertising campaigns and examines their destinations. It collected 21,541 web push notifications and identified 5,143 advertisements across 572 campaigns; 51% of the collected advertisements were reported as malicious. Traditional ad blockers and URL filters were often unable to block them.

BrowseBlock adopts examination of notification content, notification destinations, and suspicious notification-related indicators. These findings inform the browser notification-scam module, which can warn users and present supporting information through the extension. PushAdMiner was primarily a measurement and collection framework rather than a complete real-time blocking classifier. BrowseBlock adapts its detection insights without claiming to implement the entire framework.

2.9 CPDetector: Automated Detection of Excessive Permissions in Chrome Extensions, Preprint

The paper by Xiaoyu Cheng and co-authors investigates security risks associated with extensions requesting more permissions than their functionality appears to require. Extensions may access tabs, cookies, browsing history, and webpage content. Excessive permissions expose sensitive information and increase the potential for unauthorized access. The paper evaluates the relationship between declared permissions and APIs used by an extension.

CPDetector extracts permission information from `manifest.json` and analyzes JavaScript code to identify API calls corresponding to browser permissions. It compares declared permissions with permissions inferred from the implementation. PermissionGuard, a browser extension, alerts users about excessive permissions. The framework analyzed 199,058 Chrome Web Store extensions and identified 98,072, or 49.3%, as requesting excessive permissions according to its methodology. The uploaded version is a preprint and has not been peer reviewed.

BrowseBlock adopts the principle of least privilege and permission-related risk assessment for its browser-extension-abuse module. Accessible extension metadata, declared permissions, and known suspicious indicators can identify potentially risky configurations. However, a standard extension does not automatically have unrestricted access to the internal code or runtime behavior of every other installed extension. BrowseBlock must limit its claims to information available through supported browser mechanisms.

2.10 A Login Page Transparency and Visual Similarity-Based Zero-Day Phishing Defense Protocol, 2025

The paper by Gaurav Varshney, Akanksha Raj, Divya Sangwan, Sharif Abuadbba, Rina Mishra, and Yansong Gao proposes a phishing-detection and prevention protocol for fraudulent login pages. It focuses on phishing websites that imitate legitimate authentication pages and attempt to collect usernames and passwords. The authors identify limitations of URL blacklists and heuristic detection, which may miss newly created phishing pages. The proposed Page Transparency concept verifies login-page legitimacy before allowing a page to be displayed.

The approach uses a Public Log Server to maintain records of legitimate login pages and cryptographic proofs to verify registration. It also uses visual page-matching with a Siamese Network Model to identify deceptive pages that closely resemble legitimate ones. A custom HTTP Page Transparency header supports client-side verification. The authors developed the AntiPhish Chrome extension and evaluated credential-requiring pages associated with 91 brands, including 2,325 phishing pages, 115 legitimate pages, and 50 previously unseen legitimate pages.

BrowseBlock adopts login-page identification, deceptive-authentication-interface detection, and browser-level warnings for its credential-harvesting module. The module can examine accessible form fields, submission destinations, URL characteristics, and inconsistencies between displayed identity and actual domain. Visual similarity could provide an additional signal if implemented and evaluated. BrowseBlock does not require the complete Page Transparency infrastructure, including its public log server and custom HTTP header. Findings can be combined with URL analysis to warn users before they disclose sensitive credentials.

Chapter 3

Requirement Analysis

3.1 Software Requirements

BrowseBlock requirements are derived from the Software Requirements Specification. The system consists of a Chromium Manifest V3 extension and a React web dashboard supported by a FastAPI service. Mandatory behavior is expressed as a system obligation, while optional behavior depends on browser permissions, available evidence, and the selected deployment configuration.

3.2 User Classes and Characteristics

General users require short warnings, a clear recommendation, and a way to continue or return to browsing. Authorized users of the dashboard require scan forms, result history, threat evidence, and analytics. Security analysts require detailed indicators and report context for investigation. Administrators require controlled configuration and access management. These classes differ in privilege, but none should receive model credentials, database credentials, or raw data beyond the authorized workflow.

3.2.1 Functional Requirements

The system shall authenticate dashboard users through Firebase Authentication and shall verify identity at the FastAPI boundary. It shall accept supported URLs, QR codes, emails, webpages, and downloaded files for analysis. It shall select the appropriate detector from the triggering browser event or the user-selected scan type, validate input, and return a common result containing a risk level, evidence, analysis completeness, and recommended or completed protective action.

The extension shall monitor supported navigation, webpage scripts and DOM behavior, redirects, frames, notification prompts, downloads, and observable extension activity. It shall provide a warning or restriction when the browser APIs, permissions, evidence, and policy support that action. The dashboard shall allow authorized users to submit manual scans, review reports and analytics, and inspect the evidence behind a decision. The system shall never require a real password as an input to a detection workflow.

Functional Requirement Summary

The main functional requirements of BrowseBlock are:

- Authenticate dashboard users and authorize access to scans, reports, and analytics.
- Monitor browser navigation, webpages, redirects, frames, notifications, downloads, and extension activity.
- Detect URL and QR-code phishing using URL normalization, feature extraction, and classification.
- Detect reflected, stored, and DOM-based cross-site scripting indicators.
- Detect credential-harvesting forms, suspicious domains, scripts, and scam pop-ups without collecting passwords.
- Track redirect chains, evaluate final destinations, and warn about malicious navigation.
- Detect clickjacking through suspicious frames, overlays, positioning, and interaction patterns.
- Detect cryptojacking through mining scripts, WebAssembly, workers, mining endpoints, and resource behavior.
- Analyze downloaded executable, PDF, DOC, DOCX, and ZIP files using static inspection, hashes, rules, and models.
- Classify suspicious files and support warning, cancellation, quarantine, or release according to policy.

- Detect email phishing using sender details, links, authentication signals, machine learning, retrieval, and semantic analysis.
- Detect notification scams using origin, prompt text, urgency, destinations, and redirects.
- Detect browser-extension abuse through permissions, host access, injected scripts, API use, and page changes.
- Display Safe, Suspicious, or High-risk results with evidence, analysis status, and recommended protective action.
- Record detection events and clearly report unsupported inputs, unavailable evidence, failed analysis, and incomplete results.

3.2.2 Non-Functional Requirements

- **Security:** protected requests shall use HTTPS in deployment, Firebase ID tokens shall be verified server-side, and secrets shall not be exposed to the extension or dashboard.
- **Privacy:** collection shall be limited to data required for detection and authorized reporting. Email, webpage, and file content shall be submitted for deeper analysis only when permitted by the workflow.
- **Performance:** lightweight checks shall remain event-driven, while expensive models and retrieval workflows shall execute in the backend.
- **Reliability:** timeouts, unsupported formats, failed parsers, and unavailable browser events shall produce an explicit incomplete-analysis state, not a false Safe verdict.
- **Usability:** warnings shall present the risk, reason, evidence, and available action in language suitable for non-specialist users.
- **Maintainability:** detectors, models, rules, and user interfaces shall remain modular and independently testable.

3.3 Design and Implementation Constraints

The extension must operate within Manifest V3 lifecycle and permission rules. Content scripts can inspect only the pages and events exposed by the browser, and some protected or cross-origin content may be unavailable. Backend analysis depends on network connectivity, model availability, file-size limits, parser safety, and the response time of external reputation or language services. The system shall therefore use timeouts, input-size limits, isolated processing, and explicit fallback states rather than silently treating a failed operation as Safe.

3.4 External Interface Requirements

3.4.1 User Interfaces

The extension shall provide concise warnings, scan status, risk classification, indicator summaries, and an available action such as continue, cancel, or review. The dashboard shall provide authentication, URL and QR scanning, email and file submission, report viewing, analytics, and help describing analysis limitations.

3.4.2 Software Interfaces

The extension and dashboard communicate with REST-style FastAPI endpoints. The backend validates structured requests, invokes the relevant detector, and returns a consistent JSON result. Firebase Authentication supplies user identity; client components do not connect directly to protected backend resources or model files.

3.4.3 Operating Environment

The extension targets supported Chromium browsers that provide the permissions needed for the selected monitoring features. The dashboard runs in a modern browser. The backend runs in a Python environment with FastAPI and the configured model, parsing, QR-decoding, and language-analysis dependencies. A stable network is required for centralized analysis and authentication.

3.4.4 Communication and Data Interfaces

Extension and dashboard requests shall use structured JSON over HTTPS in deployment. Protected requests shall carry a Firebase ID token, while the backend shall validate the token and determine the authorized user or role. A scan request identifies the input type, source, and permitted content. A result includes the risk level, confidence or supporting evidence, analysis status, timestamps, and actual action. MongoDB stores authorized application data through backend operations; clients do not connect directly to the database.

3.5 System Features

3.5.1 URL and Fake QR Phishing

The URL detector parses protocol, host, domain, subdomains, path, query, length, symbols, IP usage, and suspicious keywords. XGBoost classifies the extracted features. A QR workflow decodes the supported image, verifies whether its payload is a URL, and passes that URL to the same detector.

The stimulus is a supported navigation event or a QR code submitted through the dashboard. The response sequence is capture or decode, normalize, feature extraction, XGBoost classification, explanation, and warning or restriction where supported. Malformed, unusually long, or encoded input must be handled without crashing. The detector shall be evaluated using precision, recall, and false-positive measurements while minimizing storage of sensitive URL parameters.

3.5.2 Cross-Site Scripting

The system inspects supported scripts, source-to-sink patterns, DOM changes, and observable requests for reflected, stored, and DOM-based XSS indicators. It reports indicators and actual enforcement separately because detection of a suspicious pattern does not prove that script execution was prevented.

The detector shall examine suspicious sources and sinks, inline scripts, event handlers, dynamically created script elements, unexpected script sources, and observable DOM manipulation. It shall distinguish reflected, stored, and DOM-based indicators where

evidence permits. Conclusions remain limited to content scripts and browser APIs visible at runtime, and legitimate page behavior must not be disrupted unnecessarily.

3.5.3 Credential Harvesting and Redirects

Credential-harvesting analysis identifies deceptive forms, password fields, mismatched page identity, external submission destinations, and suspicious scripts without collecting real credentials. Redirect analysis evaluates navigation chains, delayed JavaScript or HTML redirects, intermediary domains, frames, and the final destination where those events are visible to the browser.

For credential harvesting, the stimulus is a login or sensitive-data form. The system compares the displayed identity, registered domain, form action, password fields, external scripts, and submission destination without collecting a real password. For redirects, it records observable source and destination URLs, evaluates the final destination using URL-risk information, avoids repeated analysis of the same event, and reports when the browser did not expose a complete chain.

3.5.4 Clickjacking and Cryptojacking

Clickjacking analysis correlates invisible or deceptive overlays, iframe visibility, stacking, positioning, and interaction patterns. Cryptojacking analysis examines scripts, WebAssembly, workers, mining endpoints, and abnormal resource indicators. Legitimate overlays and resource-intensive pages require multiple indicators before a high-risk decision.

Clickjacking processing begins when a page contains frames or embedded interactive content. It examines iframe attributes, visibility, position, overlay relationships, stacking, and interaction structure, then warns or restricts only when a reliable browser-supported mechanism exists. Cryptojacking processing begins when scripts, WebAssembly, workers, or mining-related network behavior are observed. It combines configured mining-resource indicators with available resource evidence while minimizing monitoring overhead and false positives on legitimate intensive pages.

3.5.5 Malicious File Downloads

Supported downloads, including ZIP, PDF, EXE, DOC, and DOCX files, are identified from their content and extension. The workflow can calculate SHA-256 hashes, apply format-specific static analysis and deterministic rules, use reputation information, and invoke the configured model. Untrusted files are not executed merely for analysis.

When a supported download starts, the extension identifies the file and transfers only the required content or metadata to the controlled workflow. The backend applies file-specific parsers, static rules, SHA-256 and reputation checks, and XGBoost where configured. A native helper may provide controlled file access, quarantine, and release support. Size, archive, time, and resource limits shall be enforced, and the final record shall distinguish allow, warn, cancel, and quarantine actions.

3.5.6 Email Phishing

The extension performs lightweight screening using Random Forest features and available SPF, DKIM, and DMARC signals. Authorized dashboard analysis can extract URLs, retrieve relevant knowledge through RAG, and use an LLM to assess urgency, impersonation, social engineering, and credential harvesting. Extracted URLs are also evaluated by the URL detector.

The first stage screens sender and domain differences, link-text mismatch, IP-based URLs, suspicious terms, link count, script indicators, and available authentication signals. A dashboard investigation is a second stage: after user authorization, RAG retrieves context, links and patterns are extracted, the URL detector assesses extracted links, and the LLM summarizes semantic indicators. SPF, DKIM, and DMARC success is evidence only, not proof of safety. Retrieval, inference, and external requests require configurable timeouts, and failed analysis must remain visibly incomplete.

3.5.7 Browser Notification Scams and Extension Abuse

Notification analysis evaluates origin, text, urgency, fake security warnings, reward or payment language, destinations, and redirects. Extension-abuse analysis compares declared permissions, host access, observable API use, injected scripts, and page changes.

Excessive permission alone is treated as an indicator rather than proof of malicious behavior.

Notification analysis begins when a page requests permission or displays a related prompt. It correlates origin, prompt text, associated URL, fake security or prize language, and post-click behavior while minimizing interference with legitimate notifications. Extension-abuse analysis examines observable permissions, unexpected webpage tampering, suspicious injection, and host access. If browser APIs do not expose enough information, the system shall state that limitation and avoid claiming a complete assessment.

3.6 Risk Classification

Every detector maps its evidence to one of three user-facing levels: Safe, Suspicious, or High-risk. The result also records the detector category, evidence indicators, timestamp, analysis completeness, and protective action. A completed scan is not a guarantee that an item is safe, and the report must identify unsupported content, incomplete visibility, and unavailable enforcement.

3.7 Logging, Audit, and Reporting Requirements

The system shall record sufficient metadata to explain a result and troubleshoot the workflow: scan type, detector, time, status, indicators, risk level, and protective-action outcome. Sensitive URL parameters, email bodies, webpage content, and file content shall not be retained unnecessarily. Audit records shall distinguish a requested action from a completed action and shall be protected by authentication, authorization, and the configured retention policy.

3.8 User Documentation and Future Requirements

Documentation shall explain installation, supported browser versions, supported file types and limits, scan workflows, report interpretation, risk decisions, privacy behavior, and troubleshooting for failed or unsupported operations. The modular architecture shall

support future threat categories, updated models, new browser APIs, improved threat intelligence, and additional dashboard analytics. The implementation must finalize the browser/API matrix, model versions and datasets, external providers, file limits, retention periods, deployment environment, and warning, cancellation, quarantine, and release policies.

3.9 Proposed Solution

The proposed solution is an event-driven extension backed by a centralized FastAPI service and an authenticated React dashboard. The extension performs low-latency collection and user feedback. FastAPI validates input, verifies identity, dispatches detectors, coordinates XGBoost, Random Forest, static rules, QR decoding, file analysis, reputation services, and RAG/LLM workflows, and writes authorized records to MongoDB. The dashboard provides deliberate scans and deeper explanations without bypassing the backend security boundary.

3.10 Scope and Limitations

BrowseBlock is an additional browser-security layer, not a replacement for antivirus, browser security controls, or enterprise monitoring. Detection is bounded by Chromium permissions, browser API visibility, network availability, model quality, external-service availability, and the evidence observable at runtime. The first prototype will prioritize representative modules and controlled evaluation rather than claiming complete prevention of every attack.

Chapter 4

System Architecture

4.1 Architecture Diagram

The architecture diagram presents BrowseBlock as a platform composed of a Manifest V3 browser extension, a FastAPI backend, a React and Tailwind web dashboard, detection services, Firebase Authentication, MongoDB application data, and external AI, RAG, LLM, and reputation services.

The Chromium browser provides web pages, navigation events, and download events to the browser extension. The extension is the primary protection and interaction layer. It observes supported activity, collects only the evidence permitted by browser APIs and configured permissions, and sends relevant requests to the FastAPI backend over HTTPS. The extension also receives risk results and displays warnings or supported protective actions to the user.

The React dashboard provides authenticated manual scans and detailed result review. Users can submit URLs, QR codes, emails, and supported files, and can review detection results, evidence, threat reports, analytics, and authorized history. The dashboard communicates with FastAPI rather than connecting directly to detector services or MongoDB.

FastAPI is the central processing and orchestration layer. It validates request schemas, verifies Firebase ID tokens for protected operations, checks user or role authorization, selects the appropriate detector, coordinates model and rule execution, and returns a common result format. The detection-services block contains URL and QR, webpage, email, and file-analysis workflows. Expensive processing such as model inference, file parsing, RAG retrieval, and LLM analysis is handled by the backend where appropriate.

Firebase Authentication manages registration, sign-in, and identity verification. Authentication secrets and passwords are not stored in MongoDB. MongoDB stores autho-

rized application records such as user metadata, scans, detection results, evidence, threat reports, and protective-action records. Database access remains behind FastAPI, where ownership and role checks can be enforced. External AI, RAG, LLM, and reputation services are optional evidence sources called by the backend and are not exposed directly to untrusted clients.

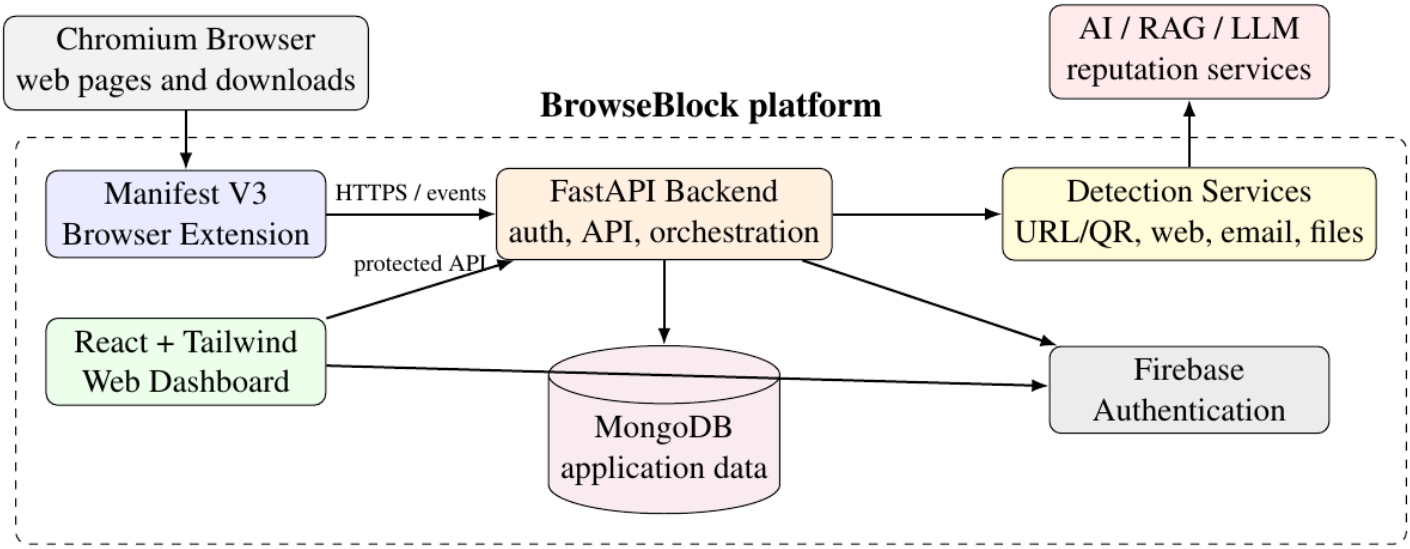


Figure 4.1: BrowseBlock architecture diagram

4.2 Activity Diagram

The activity diagram describes the complete runtime flow from browsing to detection, user decision, logging, and optional dashboard review. The workflow starts when the user opens a browser and begins browsing. The system waits for a supported event and then identifies the event type.

The "Event Detected" decision branches into URL entry, email selection, encountered QR code, malicious file download, browser-extension abuse, cross-site scripting, credential harvesting, HTML or JavaScript redirects, clickjacking, and cryptojacking. Browser notification scams appear as a related notification branch in the monitoring flow. Each event is routed to the relevant detector rather than executing every detector for every browser action.

The event branches converge before the system presents a blocking webpage or popup and evaluates the resulting evidence. The "Risk Level?" decision produces Safe, Suspicion/Suspicious, High Risk, or Inconclusive. A Safe result allows the page or activity to continue normally. Suspicious and High Risk results ask for a user action: the user may allow and load the activity or return to browsing. The Inconclusive branch displays an incomplete-analysis notification and gives the user an explicit choice instead of treating failure or missing visibility as proof that the activity is safe.

All outcome branches converge at "Log Detection Result". The logged information includes the detected category, risk level, available evidence, analysis status, timestamp, and protective action. This allows BrowseBlock to distinguish between a requested action and an action that was actually completed. The workflow then asks whether the dashboard should be opened. If the user opens it, the user can view evidence, explanations, and history. The manual URL and QR-code path at the bottom of the diagram shows that dashboard scans can enter the same reporting workflow without originating from automatic browser monitoring.

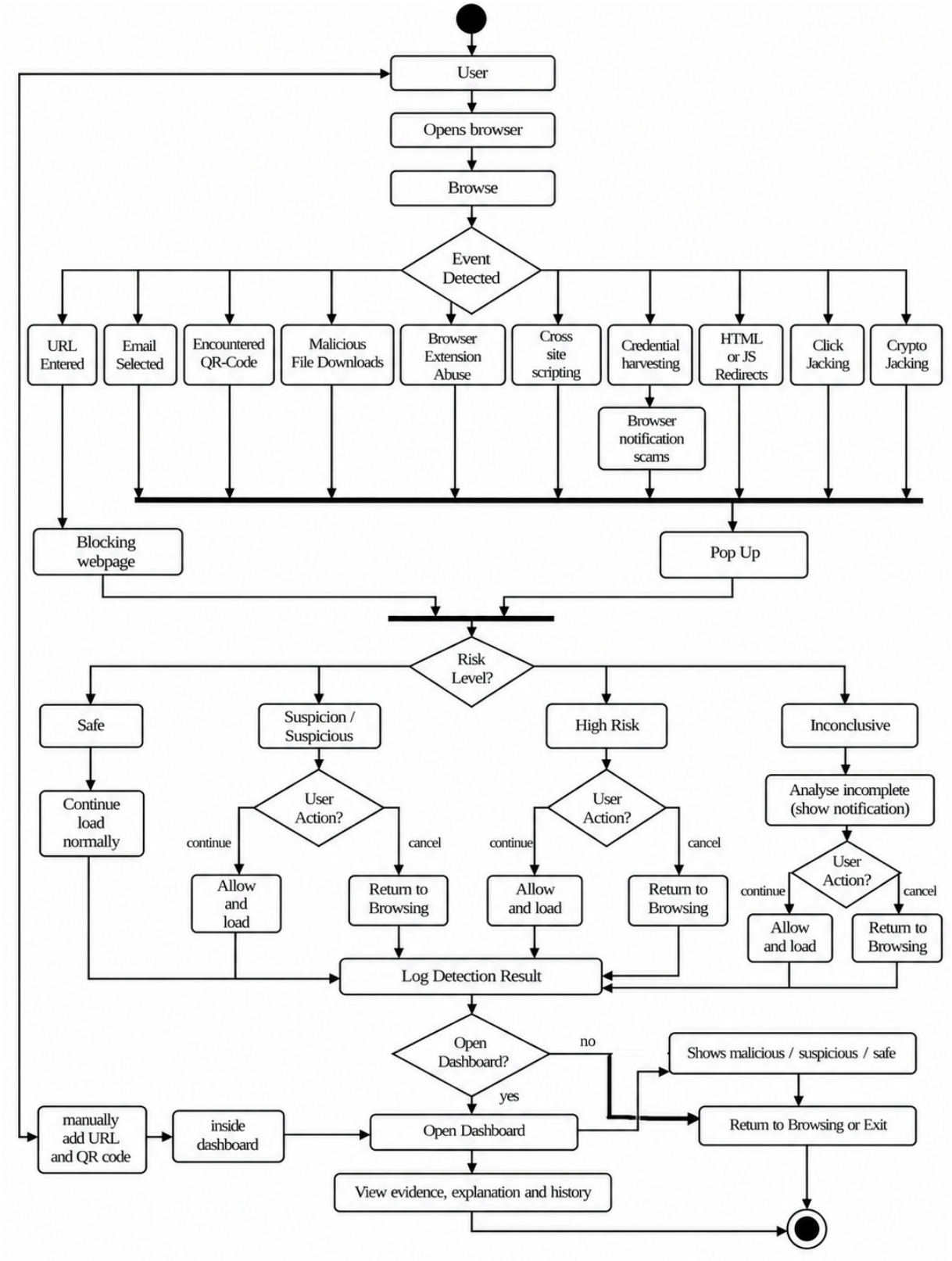


Figure 4.2: BrowseBlock activity diagram

4.3 ER Diagram

The ER diagram describes the application data created by BrowseBlock scans and security decisions. A USER can initiate zero or many SCAN records. A scan stores the input type, input reference, source, start time, completion time, and processing status. A scan can produce zero or many DETECTION_RESULT records because one input may be analyzed by more than one detector or may produce multiple threat-category findings.

Each DETECTION_RESULT stores a risk level, confidence score, summary, detection time, and analysis status. A result is associated with one THREAT_CATEGORY, such as URL phishing, XSS, malicious downloads, or email phishing, while one category can classify many results. A result can contain multiple THREAT_EVIDENCE records. Each evidence record identifies the indicator type, description, source reference, and creation time so that the dashboard can explain why a result was produced.

A detection result can trigger zero or many PROTECTIVE_ACTION records. The action type, action status, action time, and details record whether BrowseBlock attempted to warn, block, cancel, quarantine, allow, or restrict an activity and whether that action succeeded. This distinction is essential because browser permissions or unavailable APIs may prevent a requested action from being completed.

Detection results and their evidence are summarized in THREAT_REPORT records owned by an authenticated user. Reports provide the dashboard view of scan history and findings. The foreign-key relationships support traceability from a report to its scan, result, category, evidence, and protective action. FastAPI must enforce ownership and role authorization before returning these related records, while sensitive URL parameters, email content, webpage content, and file content are retained only when required by the configured privacy policy.

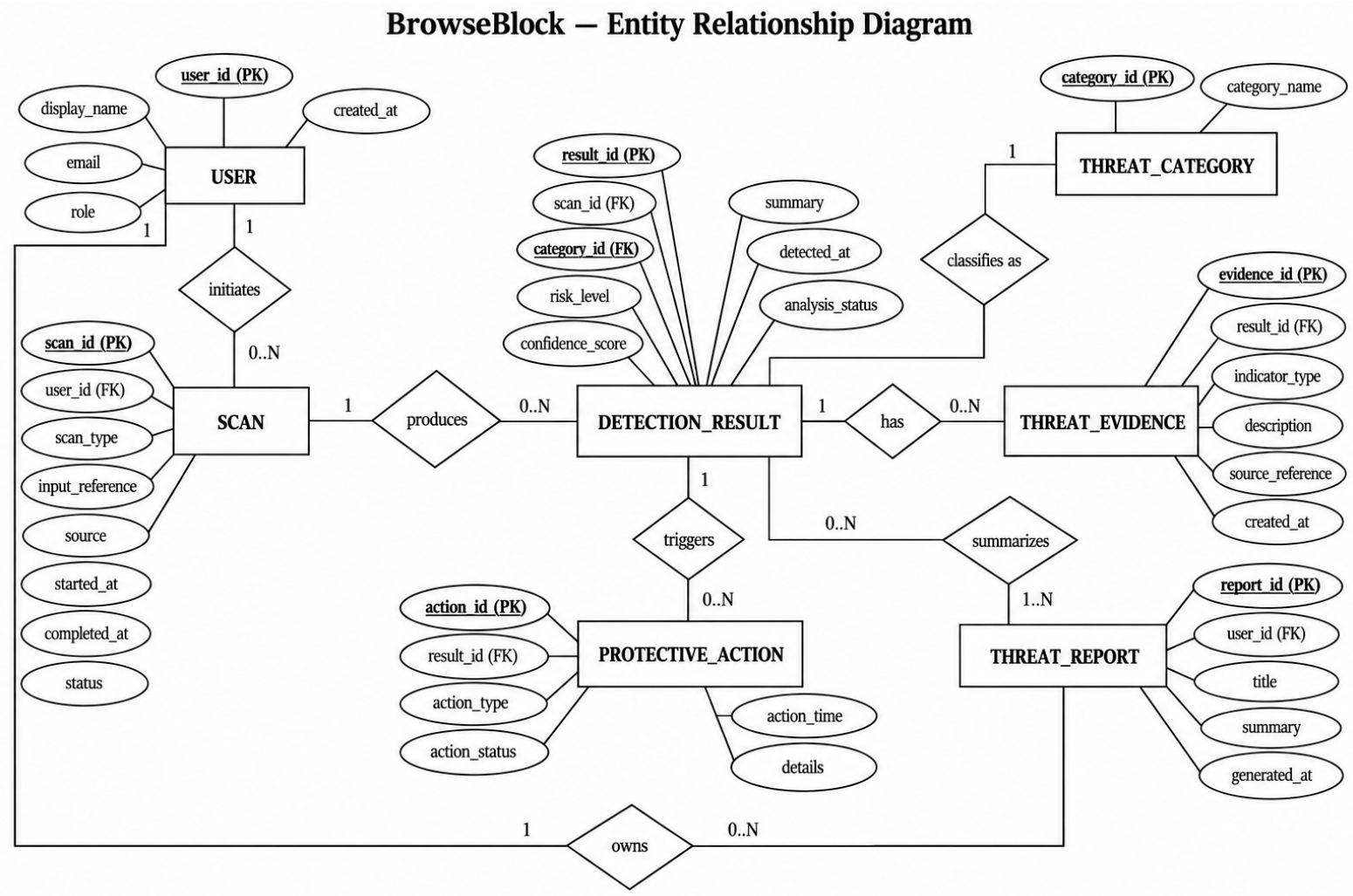


Figure 4.3: BrowseBlock entity relationship diagram

4.4 Use Case Diagram

The use-case diagram divides BrowseBlock into the Browser Extension and React Web Dashboard areas. The Web Browser actor installs or enables the extension, configures its settings, and supplies browser activity for monitoring. Monitoring includes URL phishing detection, QR-code analysis, XSS detection, credential- harvesting detection, HTML or JavaScript redirect detection, clickjacking detection, cryptojacking detection, malicious-download analysis, email-phishing detection, notification-scam detection, and extension-abuse detection.

The dashed "include" relationships show that monitoring invokes the appropriate detector for the observed event. The detector paths converge on "Block / Restrict High-Risk Activity". The associated "Display Warning / Risk Details" use case communicates the risk level, evidence, recommended response, and available protective action to the user. The "extend" relationships represent conditional behavior: restriction, blocking, or additional warning details are applied only when the evidence, configured policy, and browser capability support them.

The User actor is connected to sign-in/sign-out, dashboard viewing, detection result review, threat-detail and evidence review, detection history, analytics, and manual scanning. "Run Scan" includes separate URL, QR-code, email, and supported-file scan use cases. These dashboard actions are authenticated and processed through FastAPI so that users can inspect only records for which they have authorization.

Firebase Authentication is represented as an external actor because it validates the user's identity for dashboard access. It does not replace the FastAPI authorization boundary: the backend verifies the Firebase ID token, checks ownership or role permissions, invokes the selected detector, stores authorized records in MongoDB, and returns the result to the dashboard.

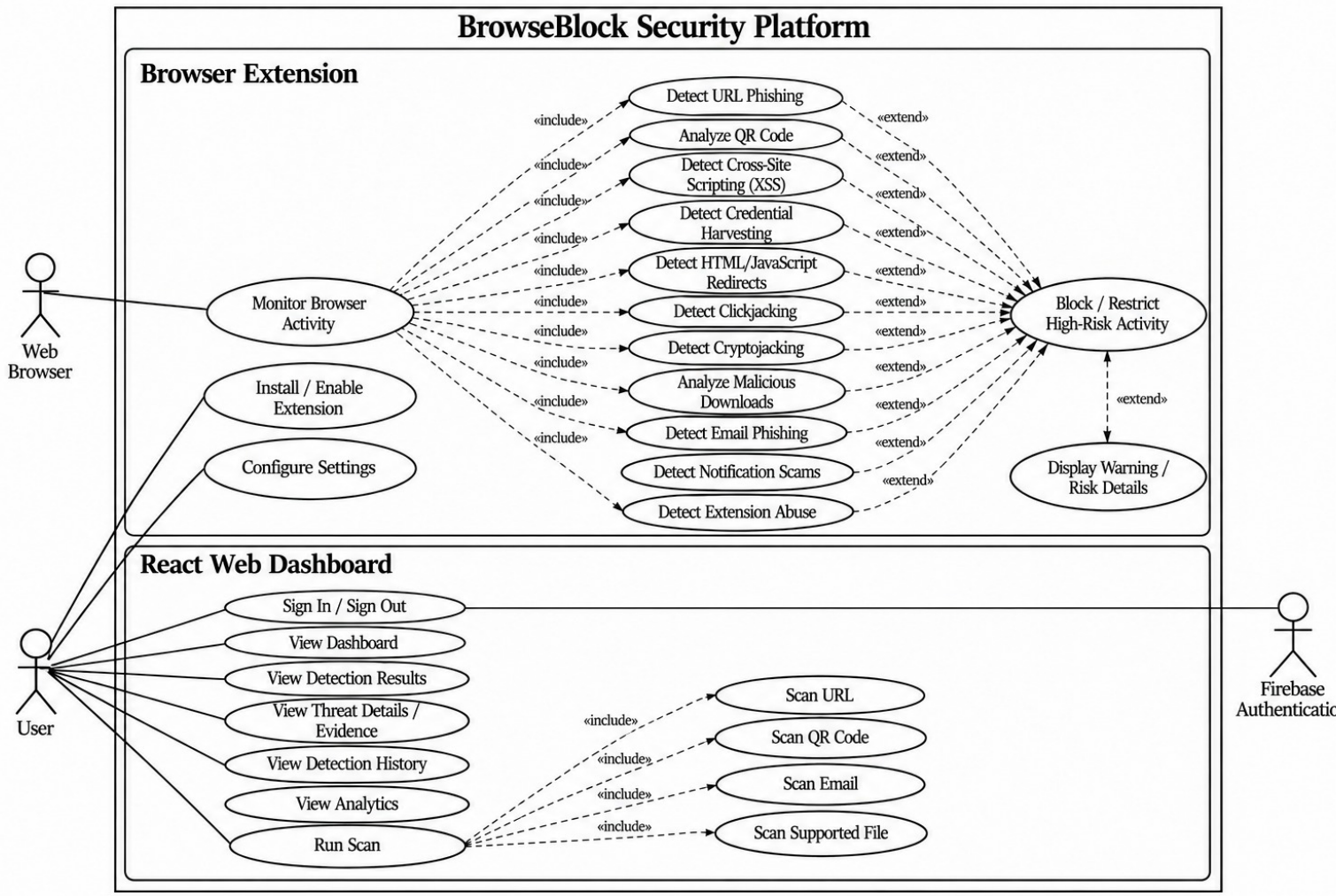


Figure 4.4: BrowseBlock use-case diagram

Chapter 5

Conclusion

BrowseBlock addresses the need for an explainable, privacy-conscious security layer for browser activity. The requirements and literature study establish a unified architecture consisting of a Manifest V3 extension, React dashboard, FastAPI backend, machine-learning and rule-based detectors, Firebase Authentication, and MongoDB application data storage behind the backend. They also define ten threat categories and clarify that warnings, blocking, quarantine, and analysis completeness must not be overstated.

The Phase I work connects the SRS requirements, literature findings, and software design into one event-driven workflow. The extension identifies the browser event, collects permitted evidence, and invokes the relevant detector instead of running all analyses on every page. The dashboard supports deeper URL, QR, email, and file investigations through FastAPI, while the common result format presents the risk, evidence, analysis status, and recommended action consistently.

The design applies clear security and privacy boundaries. Firebase Authentication manages identity, while FastAPI verifies tokens, enforces authorization, coordinates detectors, and controls MongoDB access. Passwords and authentication secrets are not used for detection or stored as application data. Results distinguish Safe, Suspicious, High-risk, and incomplete analysis, and record whether a warning, blocking, cancellation, or other protective action was actually completed.

Phase II will implement the selected detectors, validate models on representative and previously unseen data, test browser and API limitations, and measure accuracy, latency, resource usage, privacy, and confirmed protective actions. Controlled browser fixtures and synthetic credentials will support safer testing. BrowseBlock will remain an additional security layer and will communicate uncertainty whenever available evidence or browser capabilities are incomplete.

References

[1] M. Misiek and T. Hyla, "XGBoost-Based URL Phishing Detection Method With Cross-Dataset Validation," 2026.

[2] A. A. Akinyelu and A. O. Adewumi, "Classification of Phishing Email Using Random Forest Machine Learning Technique," 2014.

[3] J. Pan and X. Mao, "Detecting DOM-Sourced Cross-Site Scripting in Browser Extensions," 2017.

[4] Y. Zeng, Z. Liu, X. Chen, and T. Zang, "Hidden Path: Understanding the Intermediary in Malicious Redirections," 2022.

[5] D. Pawade, A. Lahigude, D. Reja, and E. Johri, "Implementation of Extension for Browser to Detect Vulnerable Elements on Web Pages and Avoid Clickjacking," 2016.

[6] M. Saad and D. Mohaisen, "Analyzing In-Browser Cryptojacking," 2024.

[7] N. Šrndic and P. Laskov, "Hidost: A Static Machine-Learning-Based Detector of Malicious Files," 2016.

[8] K. Subramani, X. Yuan, O. Setayeshfar, P. Vadrevu, K. H. Lee, and R. Perdisci, "When Push Comes to Ads: Measuring the Rise of (Malicious) Push Advertising," 2020.

[9] X. Cheng, Z. Wang, W. Li, W. Zeng, and X. Yang, "CPDetector: Automated Detection of Excessive Permissions in Chrome Extensions," preprint.

[10] G. Varshney, A. Raj, D. Sangwan, S. Abuadbba, R. Mishra, and Y. Gao, "A Login Page Transparency and Visual Similarity-Based Zero-Day Phishing Defense Protocol," 2025.

ANNEXURE

ST. JOSEPH'S
COLLEGE OF ENGINEERING
AND TECHNOLOGY,
- PALAI -
AUTONOMOUS

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CSD415 PROJECT PHASE I

First Review

BrowseBlock

AI Browser Extension for Real-Time Attack Detection

Date : 26-10-2026

Dept. of Computer Science & Engineering

ST. JOSEPH'S COLLEGE OF ENGINEERING AND TECHNOLOGY, - PALAI - AUTONOMOUS

Team and Guide Details

- Group No : 5

  Deon George (SJC23CS086)
  Amal Kurian Royce (SJC23CS037)
  Maria Royce (SJC23CS159)
  Melbin K Thomas (SJC23CS164)

- Project Guide

  Prof. Divya Sunny
  Assitant Professor, CSE

Contents

- Abstract
- Introduction
- Problem Statement
- Proposed Solution
- System Architecture
- Functional Requirements
- Non Functional Requirements
- Methodology
- Literature Survey
- Software Requirements
- Conclusion
- References



Abstract

- BrowseBlock is an AI/ML-powered browser security platform, made up of a Chromium browser extension and a React web dashboard, that detects, analyzes, explains and helps prevent browser-based cyber threats in real time.
- It covers ten threat categories: URL and fake QR phishing, XSS, credential harvesting, HTML/JavaScript redirects, clickjacking, cryptojacking, malicious file downloads, email phishing, notification scams and browser extension abuse.
- Lightweight, event-driven screening runs in the extension, while computationally intensive analysis (XGBoost, Random Forest, RAG + LLM, static file analysis) is handled by a FastAPI backend.
- Each result is classified as Safe, Suspicious or High-risk and explained with its evidence; scan history and threat reports are stored in MongoDB for authorized users.



Introduction

- Nowadays, attackers use automated systems to carry out web attacks, so an automated system is needed to monitor, detect, identify and prevent them. This is where BrowseBlock becomes important.
- Phishing sites, malicious scripts, scam pop-ups, unsafe downloads and abusive extensions put everyday browsing at risk for students, employees and organizations.
- Existing tools such as Google Safe Browsing, Microsoft Defender SmartScreen, VirusTotal and email spam filters each address only part of the problem.
- Many users have limited cybersecurity knowledge and need clear warnings, understandable explanations and practical recommendations.
- BrowseBlock aims to provide a unified, explainable and user-friendly security layer for individuals and organizations, without replacing existing antivirus, browser security features or enterprise security solutions.

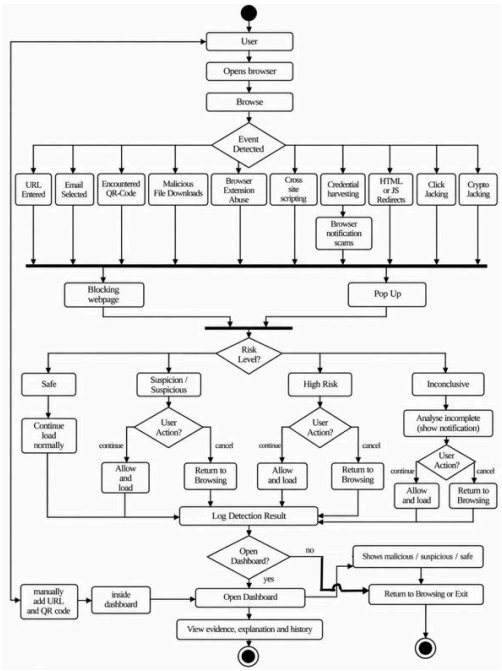
Problem Statement

- Internet users and small organizations lack a single, lightweight and explainable tool that detects and helps prevent the wide range of browser-based attacks (phishing, malicious scripts, scam prompts, unsafe downloads and abusive extensions) in real time, and that clearly tells them why a page, email or file is risky.

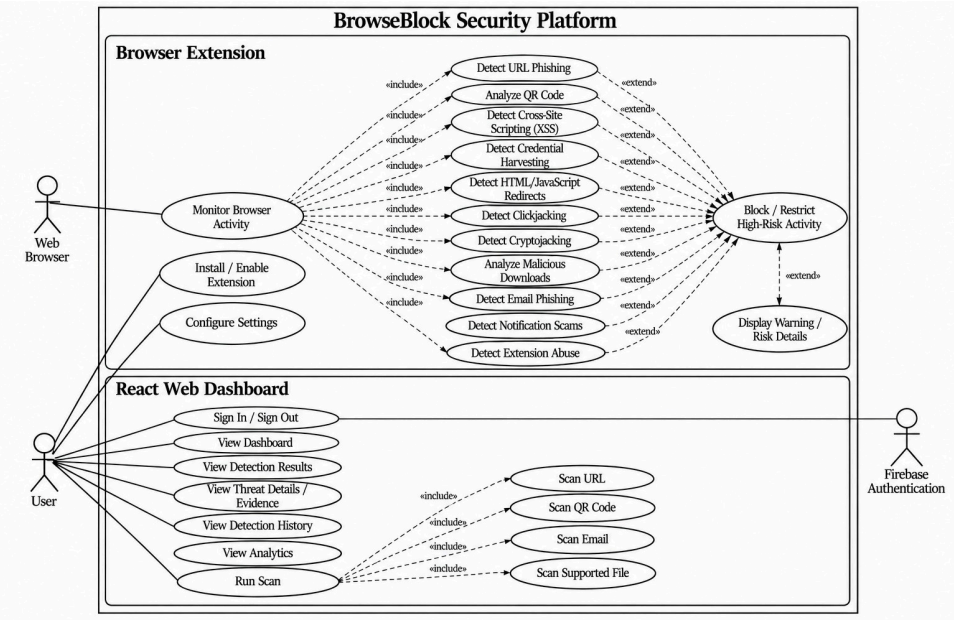
Proposed Solution

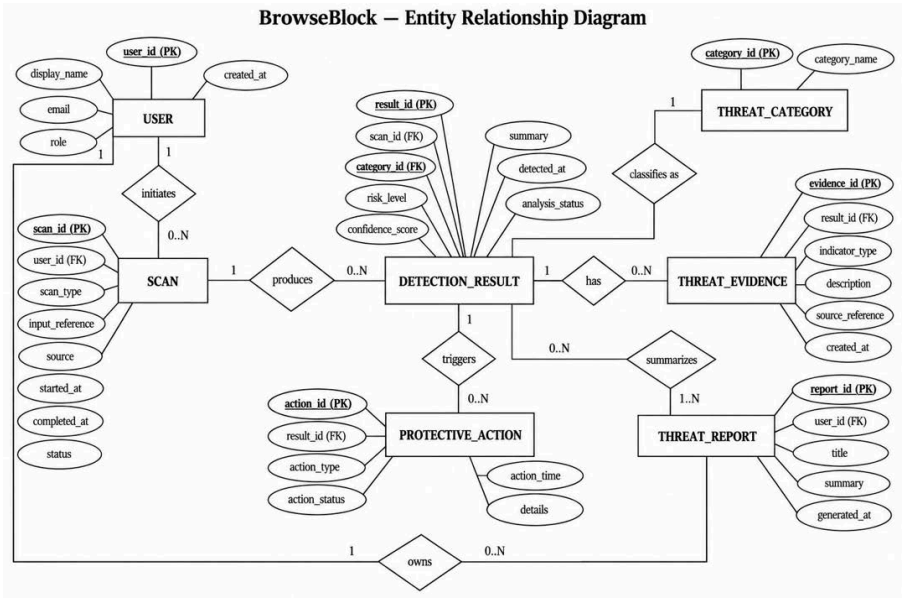
- A Manifest V3 browser extension monitors navigation, page behaviour, redirects and downloads, and warns about or blocks risky activity where browser APIs and permissions allow.
- XGBoost classifies URLs (including URLs decoded from QR codes); Random Forest with SPF/DKIM/DMARC signals screens emails in the extension.
- A RAG + LLM pipeline on the dashboard performs deeper email analysis, and extracted URLs are re-checked by the URL module.
- Downloads (.exe, .zip, .pdf, .doc, .docx) are assessed with static analysis, SHA-256 reputation checks and XGBoost, without executing the files.
- A React dashboard offers manual scans, history, analytics and SHAP-based explanations; Firebase Authentication and MongoDB protect user data.

Activity Diagram

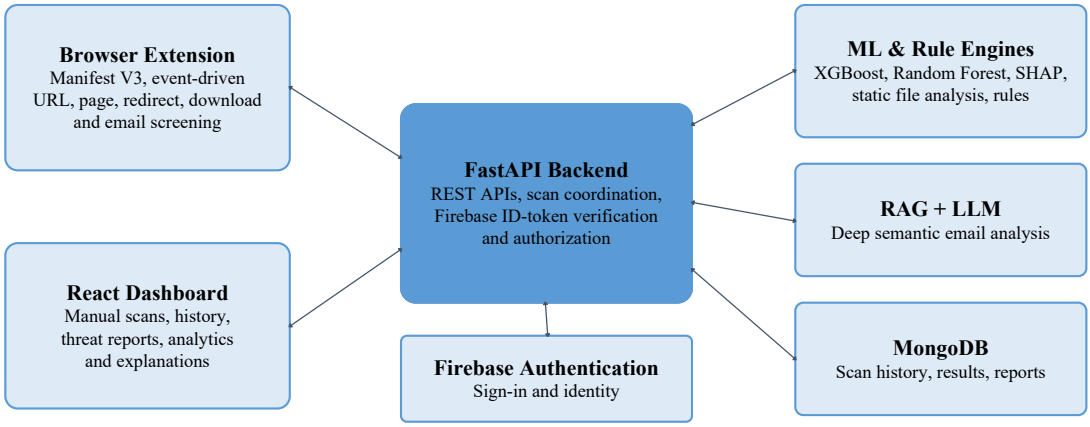


Usecase Diagram





System Architecture



Functional Requirements

- **URL & Fake QR Phishing –** normalize URLs, extract features and classify with XGBoost; decode QR codes and evaluate the embedded URL.
- **Cross-Site Scripting –** detect reflected, stored and DOM-based XSS from script patterns and source-to-sink behaviour; warn and restrict.
- **Credential Harvesting –** monitor external script domains, deceptive logins and scam pop-ups with Playwright-based analysis; block malicious resources.
- **HTML/JavaScript Redirects –** trace HTTP, JavaScript and meta-refresh redirect chains; assess the destination and interrupt malicious navigation.
- **Clickjacking –** inspect iframes, overlays and hidden elements; warn about or restrict risky interactions.

Functional Requirements cont.

- **Cryptojacking –** inspect scripts, WebAssembly, web workers and resource usage; warn about or block mining resources.
- **Malicious File Downloads –** verify file type, analyze EXE/PDF/DOC/DOCX/ZIP statically, hash and check reputation, classify with XGBoost; allow, warn, cancel or quarantine.
- **Email Phishing –** Random Forest and SPF/DKIM/DMARC screening in the extension; RAG + LLM analysis and URL extraction on the dashboard.
- **Notification Scams & Extension Abuse –** detect deceptive notification prompts and abusive extension permissions or script injection; warn the user.
- **Dashboard & Reporting –** Firebase sign-in, manual scans, detection history, threat reports and evidence behind every decision.

Non Functional Requirements

- **Performance :** lightweight, event-driven checks with minimal browsing delay; heavy analysis runs on the backend.
- **Accuracy :** models validated for precision, recall and false-positive rate; model output alone never drives high-impact blocking.
- **Security :** HTTPS, Firebase ID-token verification and ownership/role-based access to MongoDB records.
- **Privacy :** minimal collection and retention of URLs, emails, page content and files; passwords never logged.
- **Safe Processing :** untrusted files are analyzed statically, never executed, within configurable limits.
- **Maintainability :** modular detectors and model versions that can be tested and updated independently.

Methodology

- **Data Preparation:** collect labelled URL, email and file datasets (to be finalized); extract URL/domain, email and file-format features.
- **Modeling:** XGBoost for URL and file risk; Random Forest for email screening; SHAP explanations; RAG + LLM for semantic email analysis.
- **Rule-based Detection:** deterministic rules and DOM/source-sink monitoring for XSS, redirects, clickjacking, cryptojacking, notification scams and extension abuse.
- **Integration:** Manifest V3 extension, FastAPI backend with Firebase token verification, MongoDB storage, React + Tailwind dashboard.
- **Evaluation:** Playwright-based testing; precision, recall and false-positive rates measured on validation data.

Literature Survey

Feature	Existing Systems	Literature Survey	Advantages / Disadvantages	Proposed System
URL / Fake QR Phishing	Google Safe Browsing	XGBoost-Based URL Phishing Detection	Adv: no heavy GPU needed; reported to be more accurate than classical ML such as Random Forest. Dis: multiple datasets needed for training, validation and testing.	XGBoost model for URL (and decoded QR URL) classification.
HTML / JavaScript Redirects	Google Safe Browsing; Chrome webNavigation and declarativeNetRequest	Zeng et al., "Hidden Path: Understanding the Intermediary in Malicious Redirections" (2022)	Adv: reconstructs redirect chains incl. HTTP, JavaScript, HTML, delayed and multi-step redirects. Dis: large-scale crawling is too heavy for an extension; navigation monitoring, reputation checks, risk scoring and blocking are still needed.	Reconstruct HTTP, JS, HTML, delayed and multi-step redirects; webNavigation for monitoring, URL reputation for risk, declarativeNetRequest for blocking.
Cross-Site Scripting (XSS)	NoScript; Content Security Policy (CSP)	Pan & Mao, "Detecting DOM-Sourced Cross-Site Scripting in Browser Extensions"; RXSS Protect (2024)	Adv: dynamic symbolic execution detects DOM-sourced XSS; RXSS Protect targets reflected XSS in URLs. Dis: symbolic execution is expensive; RXSS Protect needs a separate ML server.	Source-to-sink analysis, reflected-XSS URL/payload analysis; add lightweight runtime DOM monitoring, local heuristics, risk-based warning/blocking and Playwright testing.

Literature Survey cont.

Feature	Existing Systems	Literature Survey	Advantages / Disadvantages	Proposed System
Browser Notification Scams	Chrome notification protection and Safe Browsing; AdGuard – Block Push API	Subramani et al., "When Push Comes to Ads: Measuring the Rise of (Malicious) Push Advertising" (2020)	Adv: analyzes notification abuse, origins, campaigns and landing URLs; shows malicious push ads can bypass ad blockers and URL filters. Dis: PushAdMiner needs instrumented browser infrastructure.	Notification-permission monitoring, page-behaviour analysis, URL reputation, selective warning/blocking and Playwright testing.
Browser Extension Abuse	Chrome Web Store / browser extension security	CPDetector: Automated Detection of Excessive Permissions in Chrome Extensions	Adv: static analysis finds excessive permissions by comparing requested permissions with actual API usage. Dis: mainly static; does not cover suspicious behaviour, page manipulation or script injection.	Static permission extraction, permission/API mismatch detection, over-privilege warnings and runtime security monitoring.
Cryptojacking	MinerBlock	"Analyzing In-browser Cryptojacking"	Adv: detects mining via WebSocket and mining-behaviour patterns; uses CPU-usage analysis. Dis: dynamic analysis can be resource-intensive in real time.	Lightweight CPU-threshold checks and WebAssembly / Web Worker detection.



Literature Survey cont.

Feature	Existing Systems	Literature Survey	Advantages / Disadvantages	Proposed System
Clickjacking	ClearClick	"Implementation of Extension for Browser to Detect Vulnerable Elements on Web Pages and Avoid Clickjacking"	Adv: detects deceptive elements through clickable-element and iframe analysis; examines visibility and cursor properties. Dis: may not detect all framing attacks.	Clickable-element, iframe, visibility/opacity, cursor, overlay-position and transparent-element analysis.
Email Phishing Detection	Email spam filters / Gmail spam protection	"A Machine Learning Approach for Phishing Email Detection Using Random Forest"	Adv: feature-based email analysis with Random Forest. Dis: conventional ML only; no RAG + LLM, URL-risk fusion or email-authentication signals.	Random Forest on URL count, suspicious URLs, length, urgency/credential keywords, HTML links and sender-domain mismatch; add SPF/DKIM/DMARC, RAG + LLM and URL-risk fusion.



Literature Survey cont.

Feature	Existing Systems	Literature Survey	Advantages / Disadvantages	Proposed System
Malicious File Downloads	VirusTotal; antivirus / endpoint scanners	Šrndić & Laskov, "Hidost: A Static Machine-Learning-Based Detector of Malicious Files," EURASIP J. Inf. Secur. (2016)	Adv: static analysis without executing files; structural features; format-specific analysis. Dis: evaluated mainly on PDF and SWF, not EXE, DOC/DOCX or ZIP; uses Random Forest; no browser interception or threat-intelligence checks.	Format-specific static analysis of EXE/PDF/DOC/DOCX/ZIP, XGBoost, rules, hash and threat-intelligence checks, download holding, allow/block decisions with logging.
Credential Harvesting	Microsoft Defender SmartScreen; Microsoft Edge	Web scraping with Playwright (2026)	Adv: HTML and JavaScript can be evaluated with Playwright. Dis: not specified in the source document.	Use Playwright to detect external domains loading scripts and block scam pop-ups.

Software Requirements

Browser: Google Chrome and compatible Chromium-based browsers (Manifest V3 extension)
Frontend: React, Tailwind CSS
Backend: Python, FastAPI, Firebase Admin SDK (ID-token verification)
Authentication & Database: Firebase Authentication, MongoDB
ML/AI: XGBoost, Random Forest, SHAP, RAG + LLM pipeline
Analysis Tools: Playwright, QR-decoding components, file-analysis libraries, optional native helper for quarantine
External Services: Threat-intelligence and file-reputation services (providers to be finalized)

VISION & MISSION OF THE DEPARTMENT

VISION

- To evolve as a school of computing with globally reputed Centre's of excellence and serve the changing needs of the industry and society.

MISSION

- The department is committed in bringing out career-oriented graduates who are industry ready through innovative practices of teaching and learning process
- To nurture professional approach, leadership qualities and moral values to the graduates by organizing various programs periodically
- To acquire self-sustainability and serve the society through research and consultancy